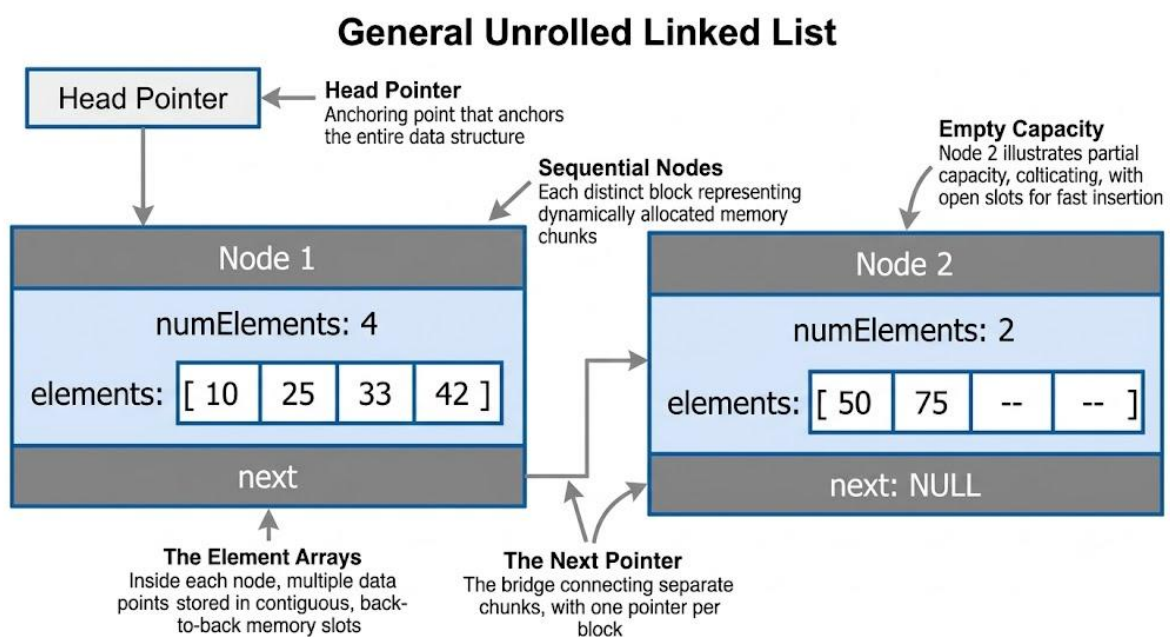


Unrolled List

–Introduction

An unrolled linked list is a hybrid data structure that bridges the gap between traditional linked lists and arrays. By combining the dynamic growth capabilities of a list with the fast, contiguous memory layout of an array, it creates a highly optimized system for managing large amounts of data.

Instead of storing just one piece of data per node, an unrolled linked list stores a fixed – size array of elements (often called a chunk or block) inside each node, alongside a single pointer directing to the next node in the chain.



Core Mechanics

- **Capacity – Based Insertion:** When new data is added, the algorithm places it into the current node's array. The list only asks the computer for new memory to create a subsequent node when the current array reaches its absolute maximum capacity.
- **Compacting Deletion:** When an item is removed, the remaining elements within that specific node's array are shifted to fill the gap. If a node becomes entirely empty, it is surgically removed from the chain and its memory is deallocated.
- **Balance Algorithms:** In advanced implementations, if a node becomes less than half – full after deletions, the list may merge it with a neighboring node to prevent having a chain of nearly empty arrays.

Primary Advantages

- **Cache Locality:** Modern CPUs load data from main memory in blocks (cache lines). Because an unrolled list stores data sequentially in arrays, the CPU can load multiple elements into its high – speed cache at once. This makes searching and traversing dramatically faster than jumping to random memory

addresses for every single element in a standard linked list.

- **Reduced Pointer Overhead:** *In a traditional linked list, a 64 – bit system wastes 8 bytes on a pointer for every single integer stored. Unrolled lists slash this overhead by sharing a single pointer across multiple elements (e. g., 1 pointer per 4, 64, or 256 elements).*
- **Dynamic Growth:** *Unlike standard arrays that suffer from expensive, massive memory reallocations when they run out of space, unrolled lists simply attach a new lightweight node to the end of the chain.*

Trade – offs and Limitations

- **Shift Penalties:** *While finding data is fast, inserting or deleting an item in the middle of an array chunk requires shifting the adjacent elements left or right.*
- **Internal Wasted Space:** *If a system heavily deletes items and nodes are left only partially full, the unused array slots within those nodes consume memory that isn't actively storing data (internal fragmentation).*